



西安电子科技大学
XIDIAN UNIVERSITY

《操作系统》

第三章 存储管理



人工智能学院

考纲内容

❖ 内存管理

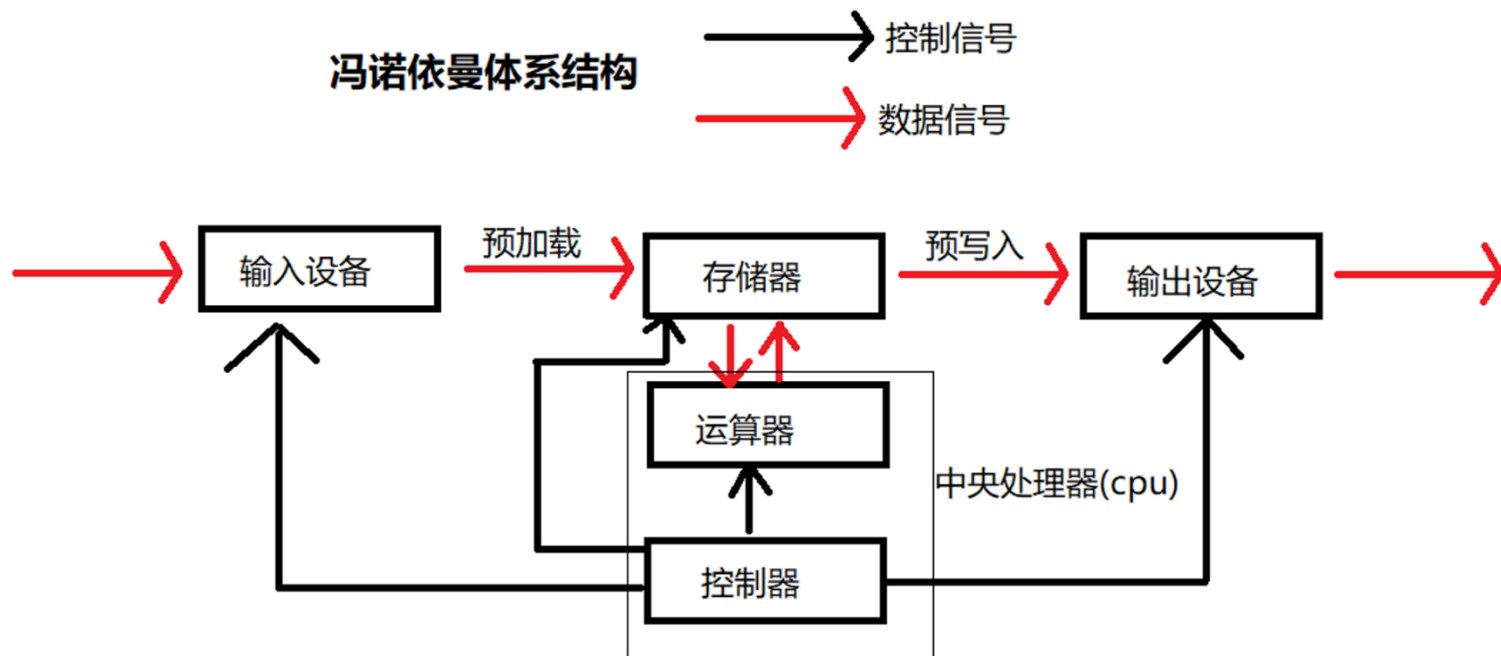
（一）内存管理基础内存管理概念；逻辑地址与物理地址空间，地址变换，内存共享，内存保护，内存分配与回收连续分配管理方式；页式管理；段式管理；段页式管理

（二）虚拟内存管理虚拟内存基本概念；请求页式管理；页框分配与回收；页置换算法内存映射文件（Memory-Mapped Files）；虚拟存储器性能的影响因素及改进方式

【复习提示】内存管理和进程管理是操作系统的核心内容，需要重点复习。本章围绕分页机制展开：通过分页管理方式在物理内存大小的基础上提高内存的利用率，再进一步引入请求分页管理方式，实现虚拟内存，使内存脱离物理大小的限制，从而提高处理器的利用率。

引言

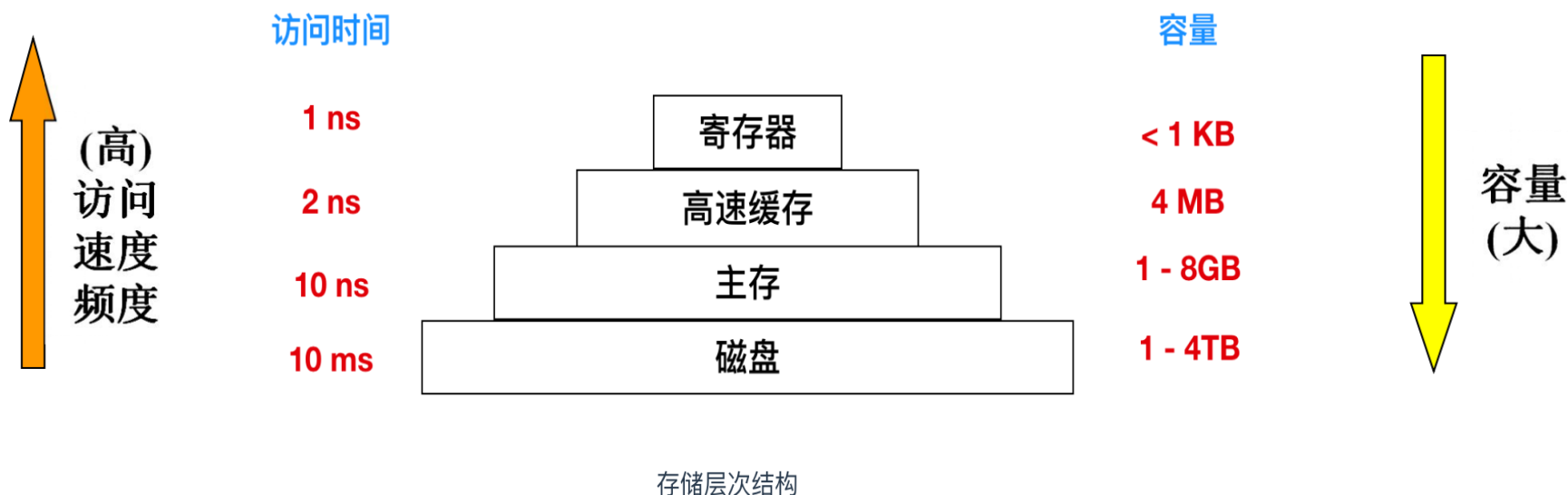
❖ 操作系统是管理者，存储是其管理的一个重要资源



◆ 程序必须先加载到存储器中，然后被执行

引言

- ✚ 存储器是计算机体系结构的重要组成部分，其功能是保存指令和数据。
- ✚ 存储层次架构(典型的5级):

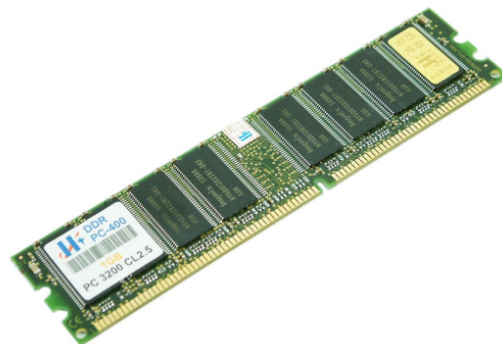


引言



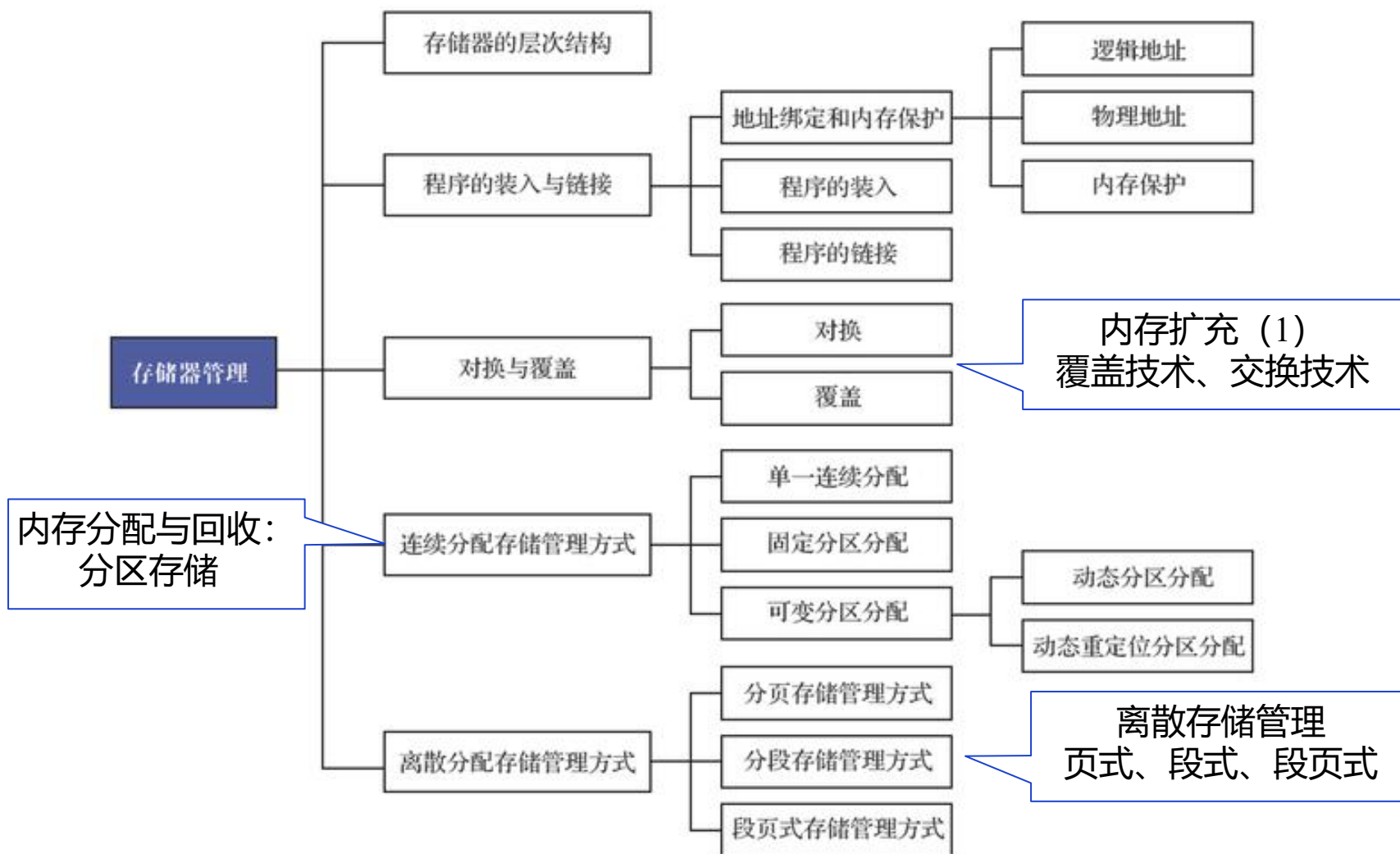
存储管理的目的

- ❖ 方便使用者
 - 屏蔽物理内存细节
 - 完成程序的自动装载
- ❖ 有效利用存储资源，提高系统效率
 - 提高内存利用率
 - 解决速度矛盾
 - 实现内存共享



- ✓ **地址保护：**一个程序不能访问另一个程序的地址空间
- ✓ **地址独立：**程序发出的地址与物理主存地址无关

存储管理



存储管理

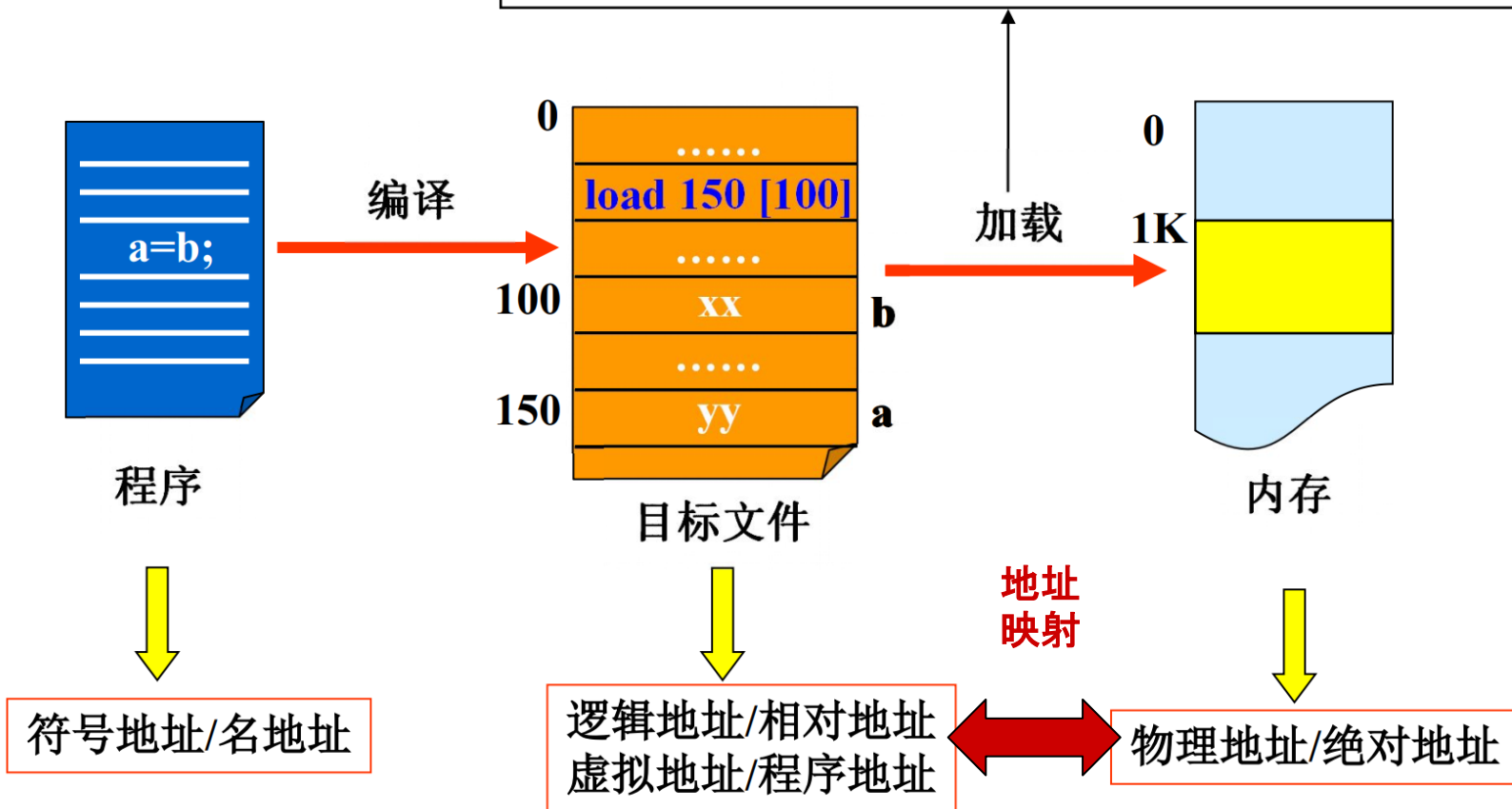
- 一. 地址重定位
- 二. 内存扩充 (1)
- 三. 内存分配与回收
- 四. 页式存储管理
- 五. 段式存储管理
- 六. 段页式存储管理
- 七. 内存扩充 (2)

一、地址重定位



基本概念

定义：当程序被装入内存时，程序的**逻辑地址**被转换成内存的**物理地址**，这一过程称为**地址重定位**。这一过程由内存管理单元(MMU)完成。

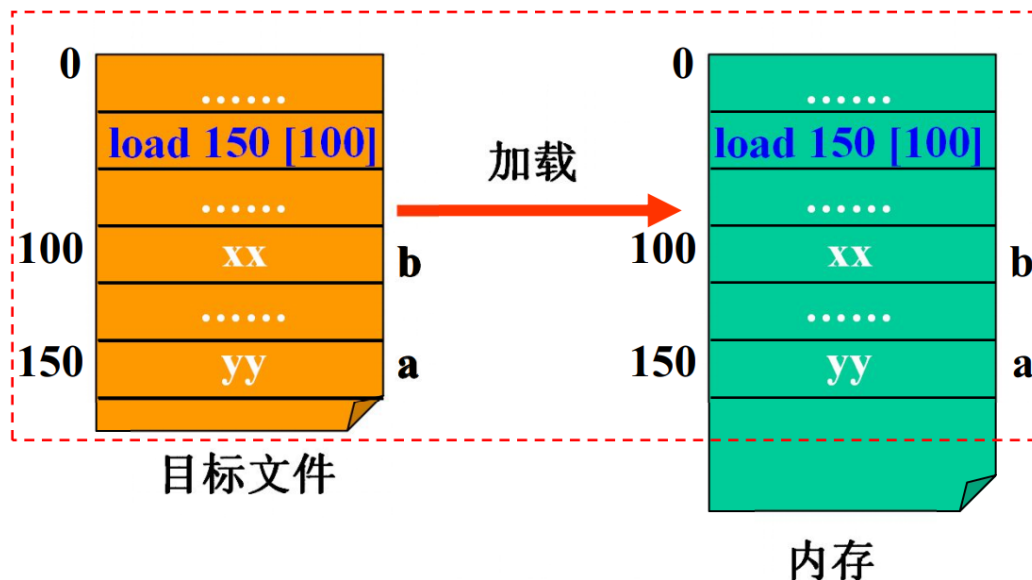


一、地址重定位

常见的地址重定位技术

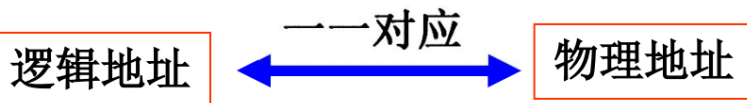
❖ 绝对装入(Absolute loading) / 固定地址再定位

- 程序的地址再定位是在程序执行之前被确定的，也就是在编译链接时直接生成实际存储器地址(物理地址)。在此，程序地址空间和内存地址空间是一一对应的。



例如：单道环境系统，单片机，MS-DOS中`.com`格式程序。

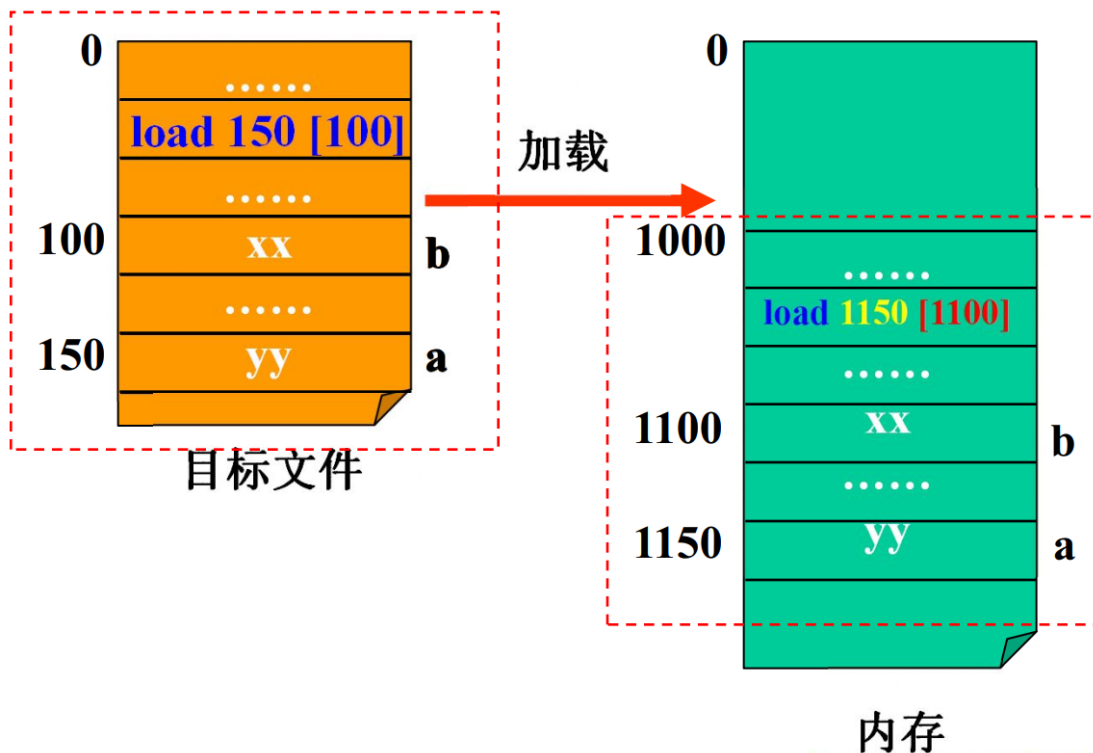
优点：装入过程简单。
缺点：与硬件的结构过于密切，缺乏灵活性。



一、地址重定位

✚ 可重定位装入(Relocatable Loading)

- ❖ 程序装入内存时，逻辑地址和物理地址可以不一致，由此需要进行逻辑地址到物理地址的映射。
- ❖ **静态再定位**：指地址定位时修改程序的逻辑地址值，完成定位后，在程序的执行期间地址将不再发生变化。特点：在程序**执行之前**进行地址再定位。

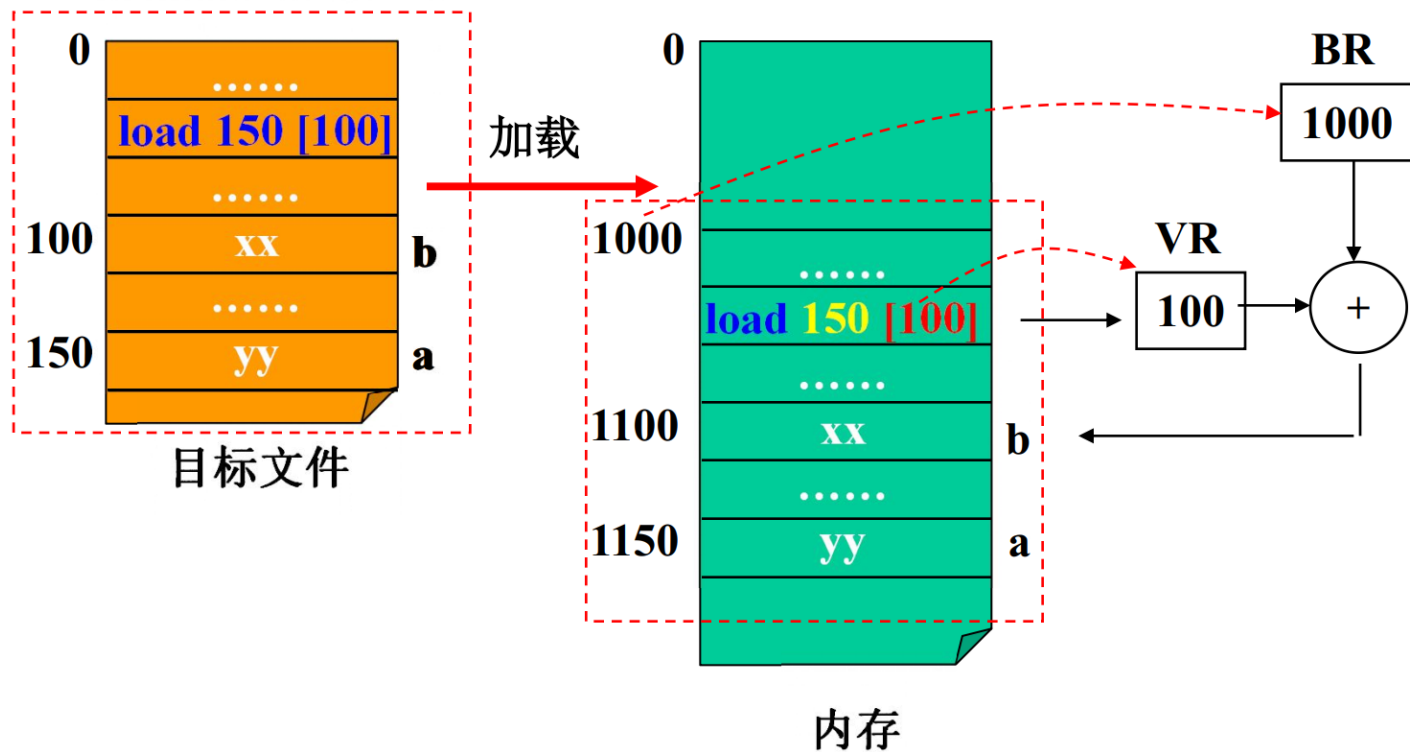


优点：容易实现，无需硬件支持。早期的操作系统中大多数都采用这种方法。

缺点：必须分配连续的存储区域；执行期间不能在内存中移动。

一、地址重定位

- ❖ **动态再定位：**程序在装入内存时，不修改程序的逻辑地址值，程序在访问物理内存之前，再实时地将逻辑地址转换成物理地址。



优点：代码可以在内存中移动，代码可以不连续存放在内存。
缺点：需要附加硬件支持，实现存储管理的软件算法比较复杂。

内存管理内容提要

- 交换和覆盖技术
- 分区存储管理方案
- 页式存储管理方案
- 段式存储管理方案
- 段页式存储管理方案
- 虚拟存储管理方案

二、内存扩充 (1)

提出原因

- ❖ 在基本的存储管理系统中，当一个作业的程序地址空间大于内存可以使用的空间时，该作业就不能装入运行，并发运行进程数受到了内存空间的限制。

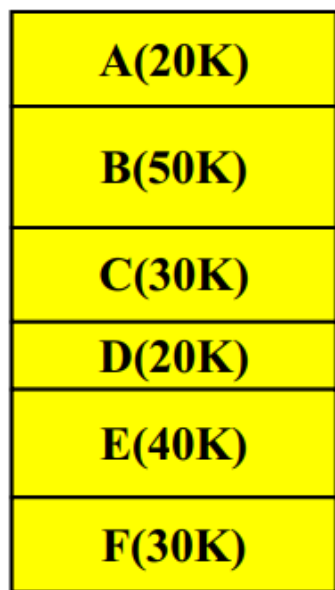
内存扩充技术

- ❖ 就是借助大容量的辅存在逻辑上实现内存的扩充，来解决内存容量不足的问题。

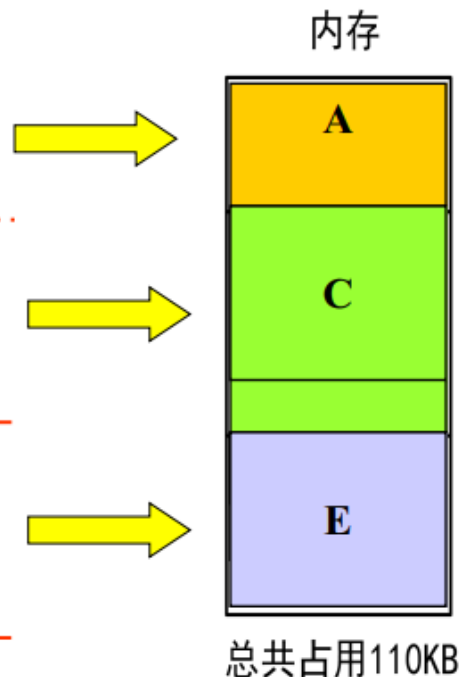
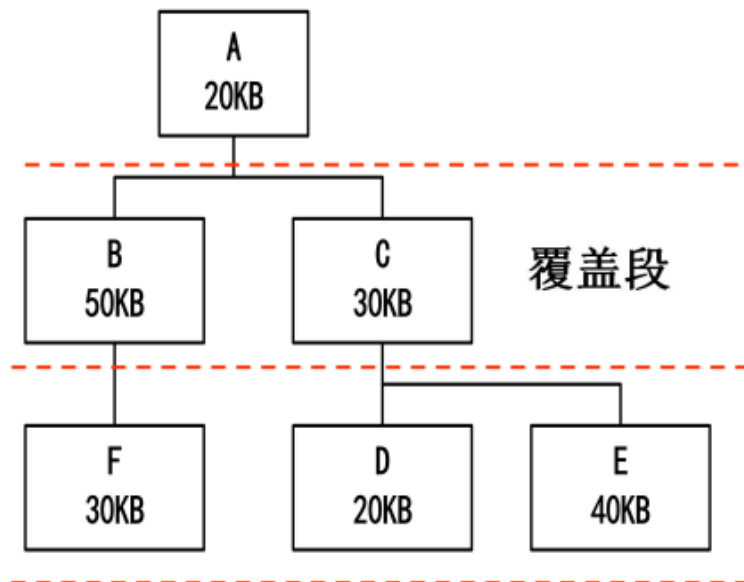
二、内存扩充 (1)

覆盖技术(Overlay)

❖ 原理



程序(190K)



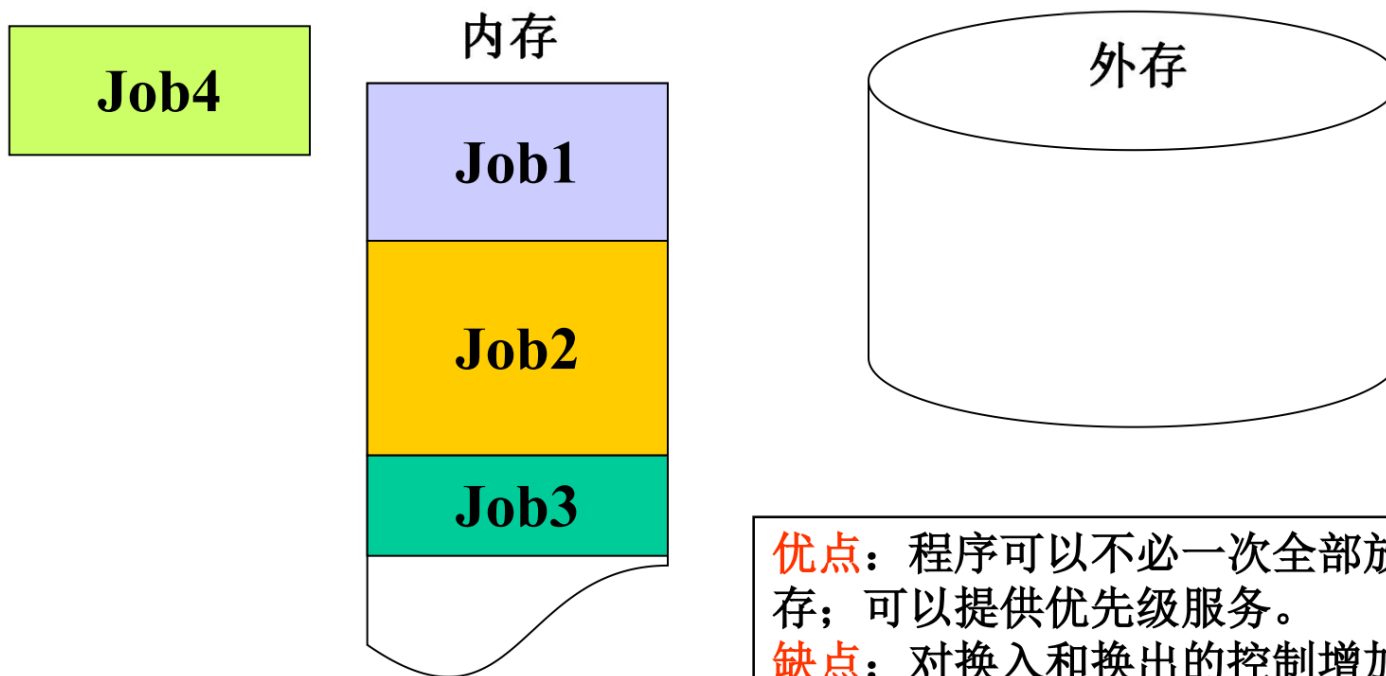
优点: 程序可以不必一次全部装入。
缺点: 覆盖结构需要程序员在程序编写的时候精心安排, 并用覆盖描述语言描述, 增加编程复杂度; 覆盖区仍然存在着碎片。

二、内存扩充 (1)



交换技术(swapping)

- ❖ 最早应用于MIT开发的CTSS中。
- ❖ 原理



优点：程序可以不必一次全部放入内存；可以提供优先级服务。

缺点：对换入和换出的控制增加处理机开销；程序换入时的重定位问题。

二、内存扩充（1）



交换技术和覆盖技术的区别

内容	覆盖技术	交换技术
适用情况	作业内部	作业之间
对程序结构的影响	有	无

三、内存分配与回收—分区（连续）存储管理方案

✚ 分区存储管理方案是一种连续分配存储空间的管理方式。曾被广泛地应用于1960~1970年代的操作系统中。

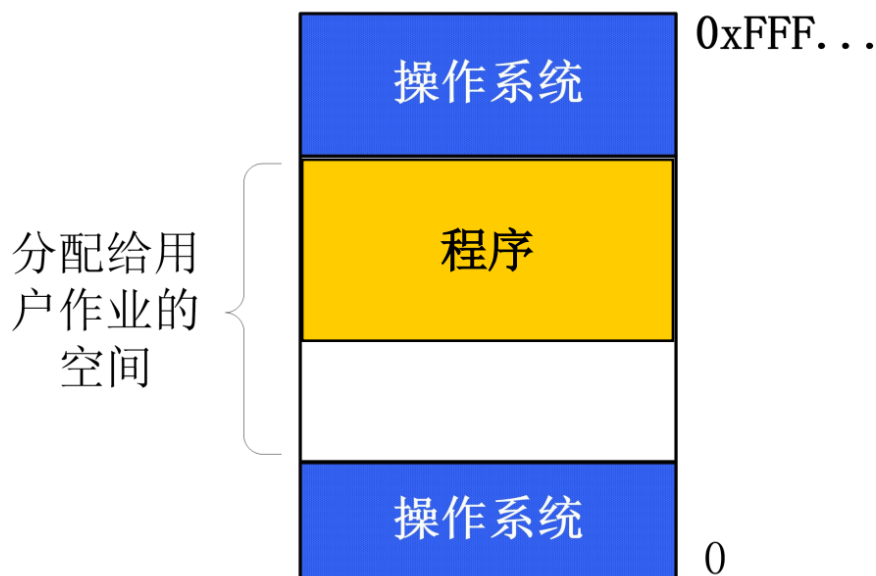
✚ 分类：

- ❖ 单一连续分区存储管理
- ❖ 固定分区管理
- ❖ 可变分区管理

三、内存分配与回收—分区（连续）存储管理方案

✚ 单一连续分区存储管理(Single partitioning)

- ❖ 特点：内存中一次只能装入一个用户程序，程序独占整个用户区，如果程序小于用户区，则剩余的空间浪费，如果大于，则无法装入。



优点：简单，适用于单用户、单任务的操作系统，不需要复杂的硬件支持。

缺点：一个作业运行时要占用整个内存地址空间，对内存造成了很大的浪费，不支持大作业。

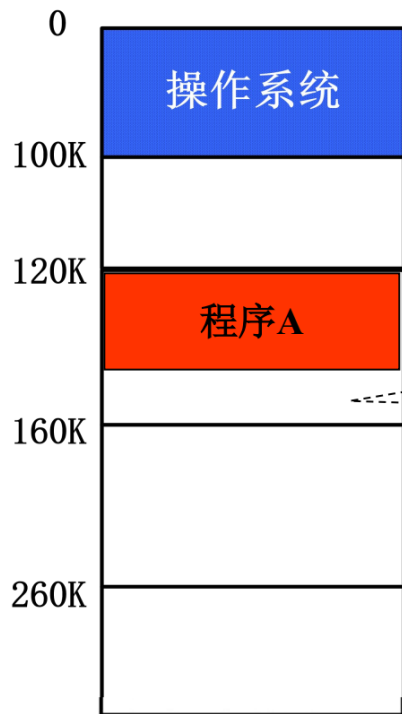
三、内存分配与回收—分区（连续）存储管理方案

✚ 固定分区管理(Fixed partitioning)

❖ 实现方法:

分区状态表

分区号	大小	地址	状态
1	20KB	100KB	未分配
2	40KB	120KB	已分配
3	100KB	160KB	未分配
4	200KB	260KB	未分配



初始化内存空间

内碎片：指占用分区之内未被利用的空间。



程序A(30K)

优点：内存的利用率提高了；可以支持多道程序。

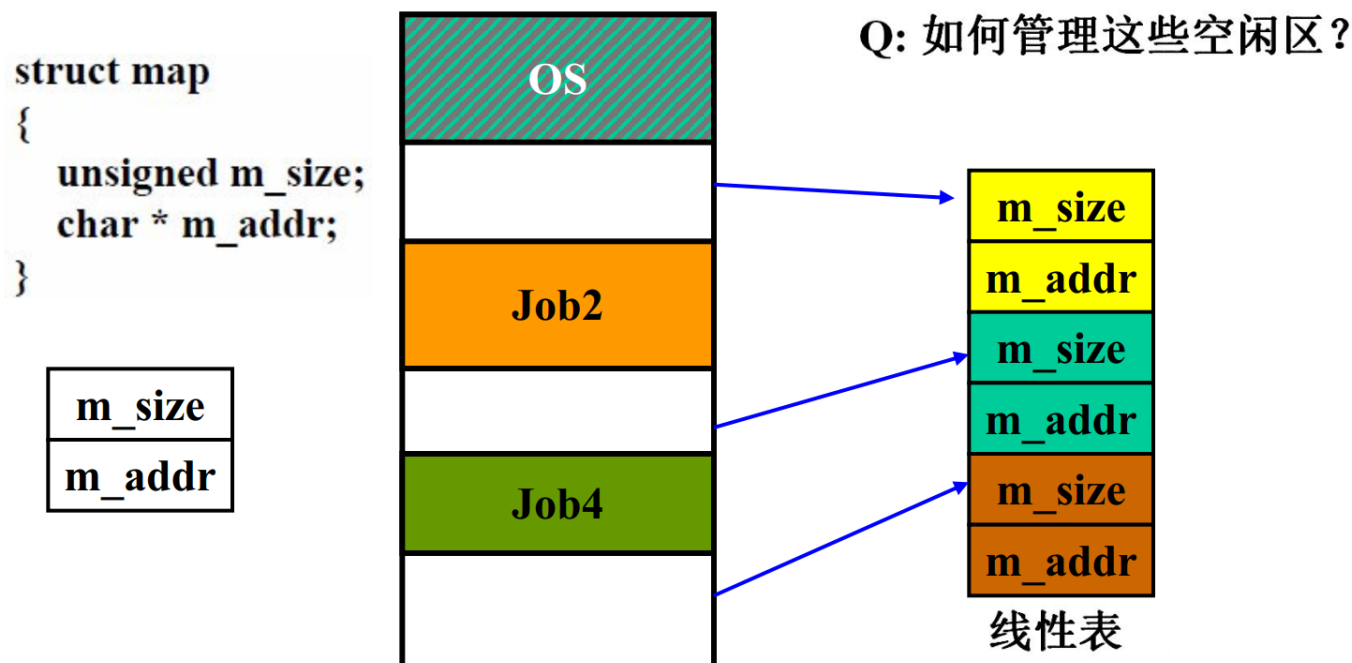
缺点：存在内碎片，造成存储空间浪费；分区总数固定，限制了并发执行的程序数目。

三、内存分配与回收—分区（连续）存储管理方案

✚ 可变分区(Dynamic Partitioning)

- ❖ 思想：预先不划分内存，当作业需要装载时向系统申请内存空间，系统从其中挖出一块给该作业，其大小等于作业所需内存的大小，然后将剩下的部分再作为空白块给下一次分配使用。

Job5

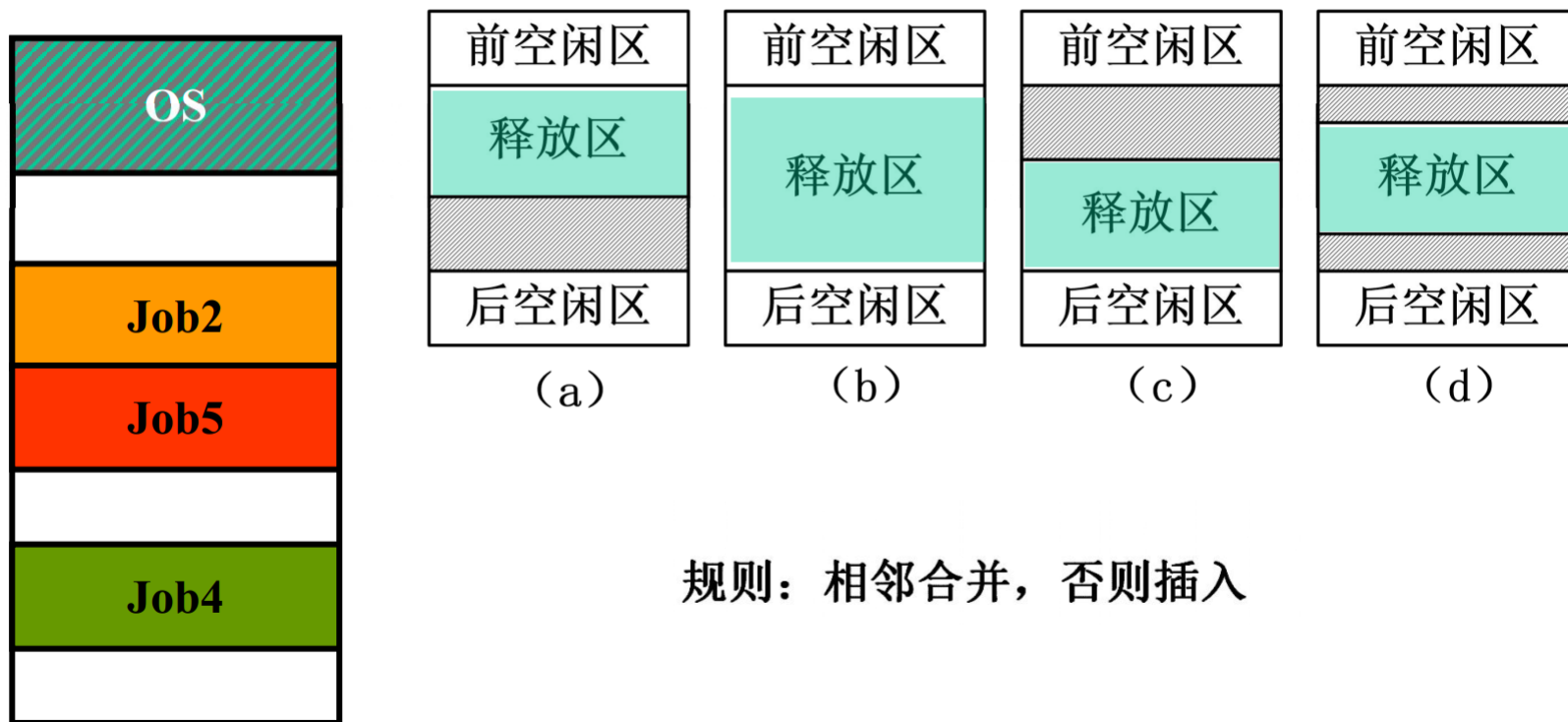


三、内存分配与回收—分区（连续）存储管理方案

三、内存分配与回收—分区（连续）存储管理方案

可变分区—>最先适应算法

➤ 释放方法



三、内存分配与回收—分区（连续）存储管理方案

优点：尽可能利用存储区低地址的空闲区，而在高地址部分保存较大的空闲区，容易满足大作业；在释放内存分区时，如果有相邻的空白区就进行合并，使其成为一个较大的空白区。

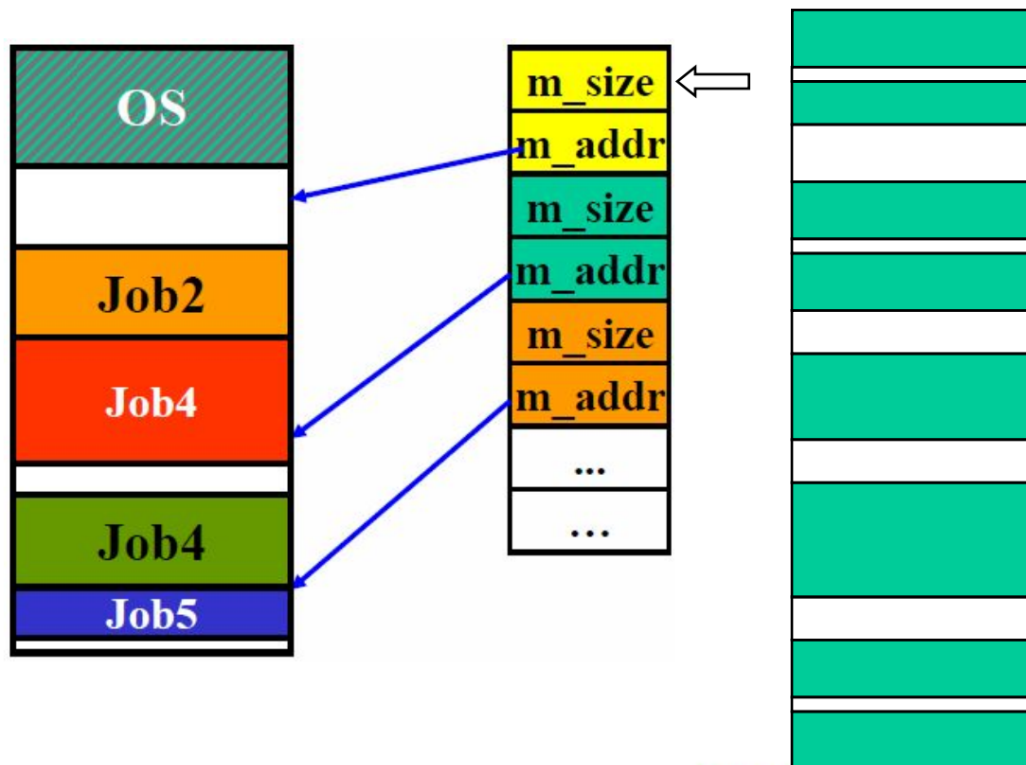
缺点：查找总是从表首开始，因此前面的空闲区往往被分割得很小时，查找次数增大。会产生外碎片(指占用的分区之间难以利用的空闲分区)，这些碎片散布在存储器的各处，不能集中使用，因而降低了存储器的利用率。



三、内存分配与回收—分区（连续）存储管理方案

❖ 下次适应算法(next-fit, 循环最先适应算法)

- 分配方法：按分区的先后次序，从上次分配的分区起查找，到最后分区时再回到开头，符合要求的第一个分区就是找到的分区。
- 释放方法：同于最先适应算法。



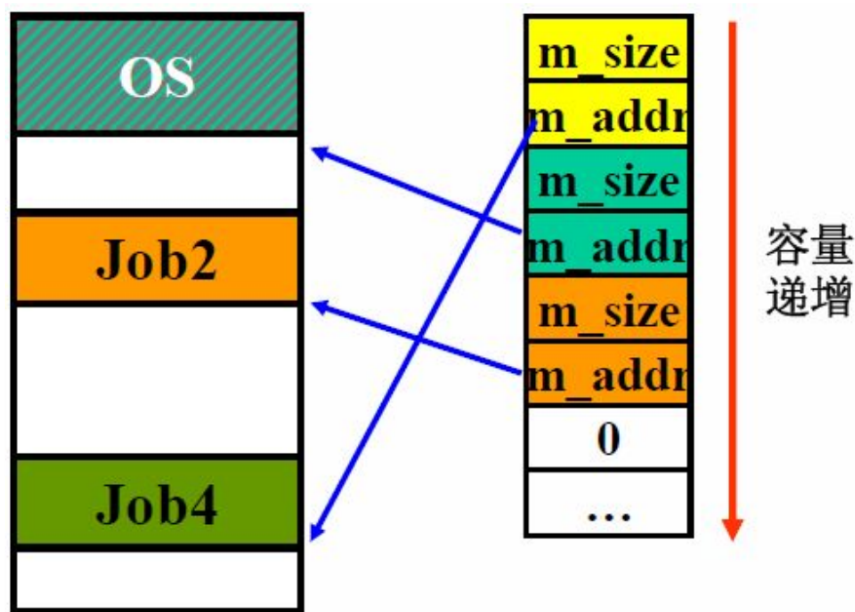
优点: 使空闲分区分布得更均匀, 提高了分配查找的速度。

缺点: 较大的空闲分区不易保留。

三、内存分配与回收—分区（连续）存储管理方案

❖ 最佳适应算法(best-fit)

- 分配方法：将所有的空闲分区按照其容量递增的顺序排列，当要求分配一个空白分区时，由小到大进行查找，找到最合适的分配。
- 释放方法：在整个链表上搜索地址相邻的空闲区，合并后，再插入到合适的位置。



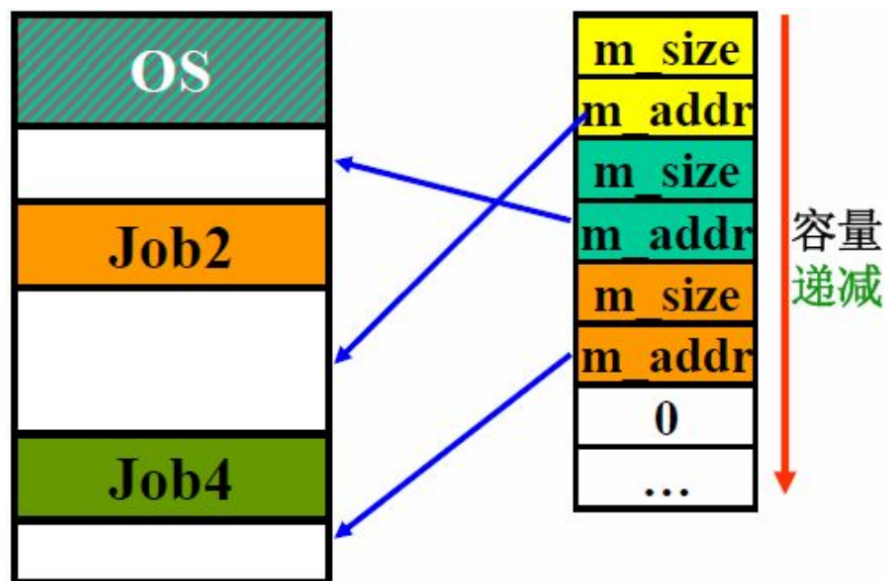
优点：如果有一个空白区的容量正好满足要求，则它必被选中。分配后所剩余的空白块会最小，较大的空闲分区会被保留。

缺点：空白区一般不可能恰好满足要求，在分配之后的剩余部分通常非常小，以致小到无法使用，会形成较多碎片。

三、内存分配与回收—分区（连续）存储管理方案

❖ 最坏适应算法(worst-fit)

- 分配方法：与最佳适应算法相反，将所有的空白分区按容量递减的顺序排列，最前面的最大的空闲分区就是找到的分区。
- 释放方法：同于最佳适应算法(best-fit)



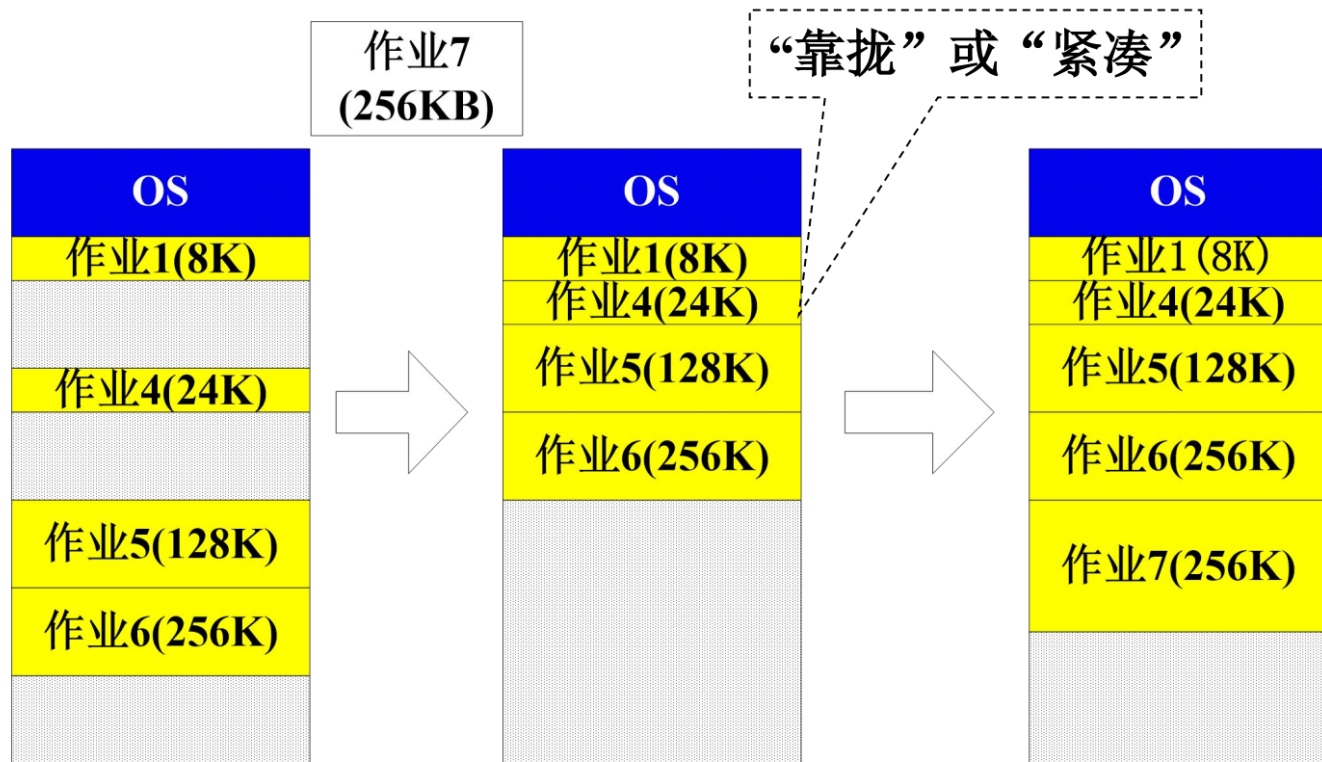
优点：分配的时候，只需查找一次，就可以成功，分配的算法很快。

缺点：最后剩余的分区会越来越小，不会保留较大的空闲分区，无法运行大程序。

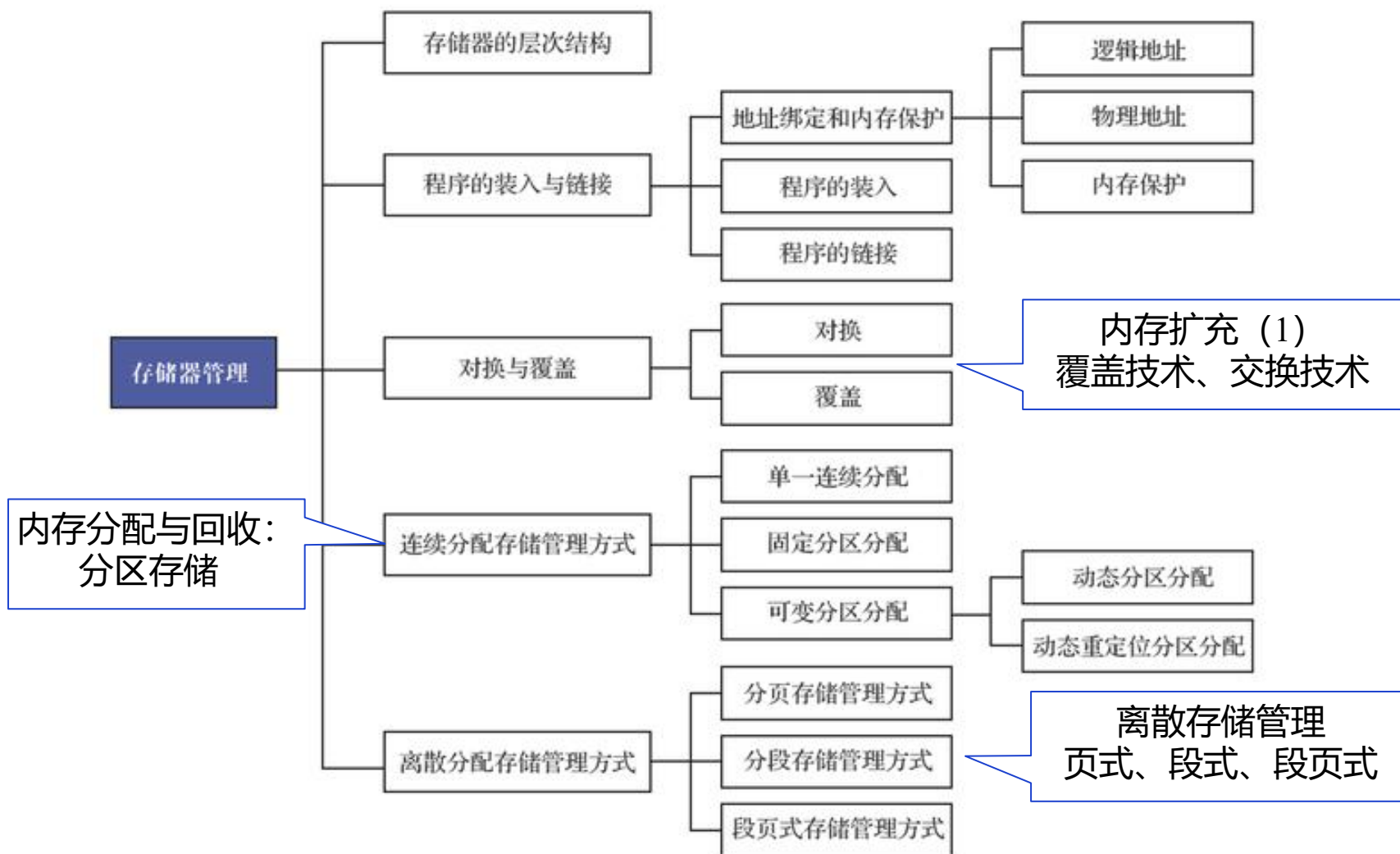
三、内存分配与回收—分区（连续）存储管理方案

✚ 可再定位式分区

- ❖ 又称浮动分区分配，是解决碎片问题的简单而有效的办法。
- ❖ 基本思想：移动所有被分配的分区，使之成为一个连续区域，而留下一个较大的空白区。



回顾——存储管理



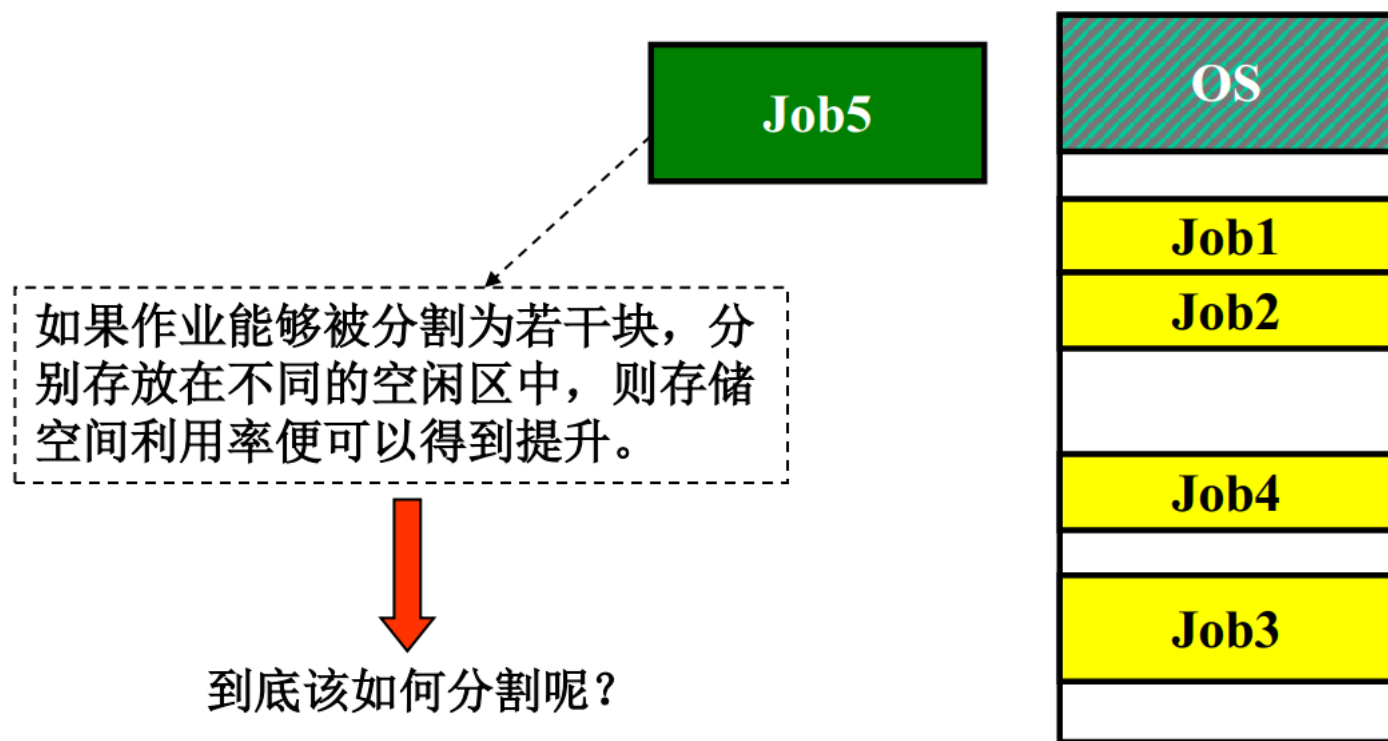
内存管理内容提要

- 交换和覆盖技术
- 分区存储管理方案
- 页式存储管理方案
- 段式存储管理方案
- 段页式存储管理方案
- 虚拟存储管理方案

四、页式存储管理

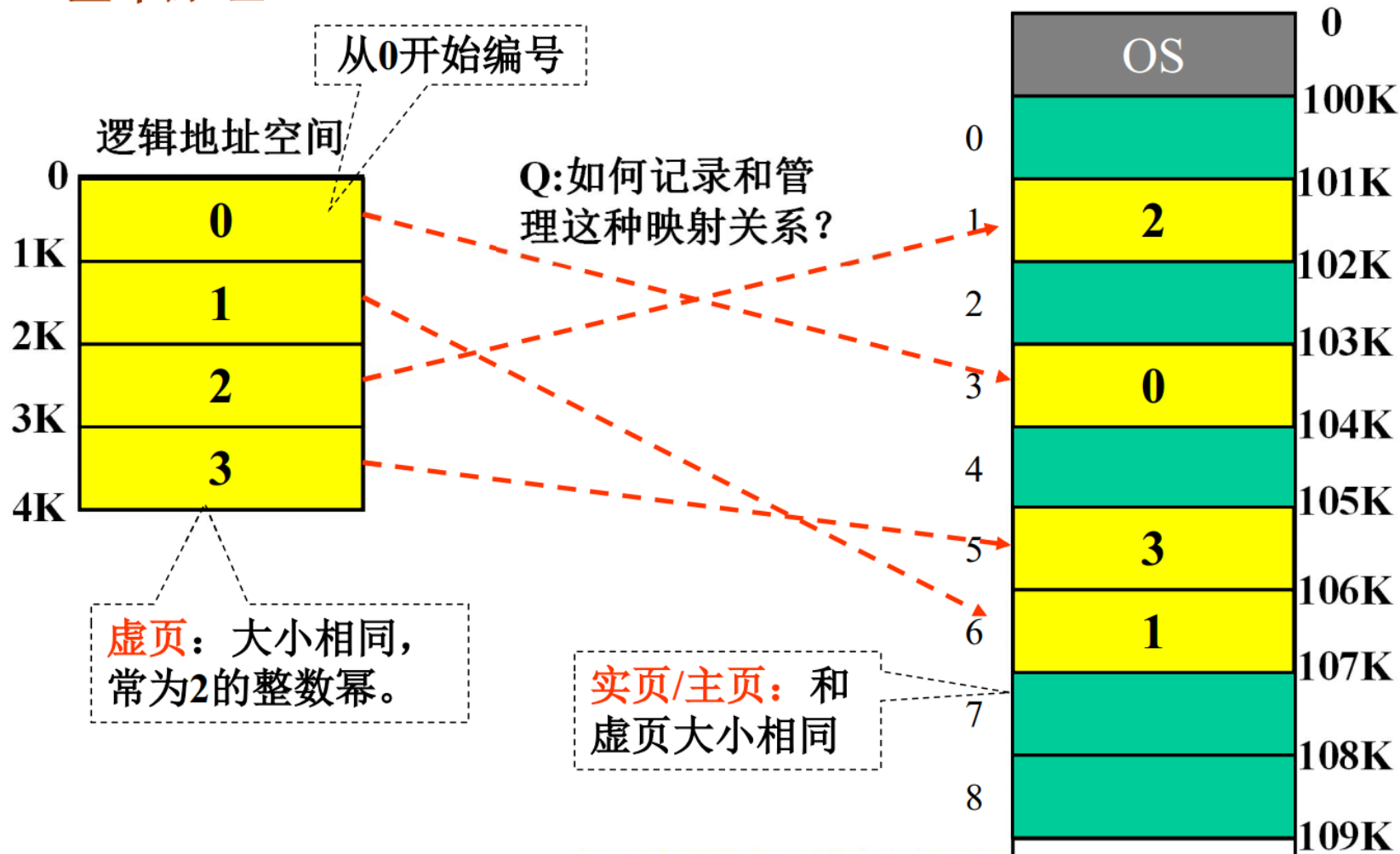
提出原因

- ❖ 分区存储管理方案要求：作业存储时必须连续存放。这样当一个作业大于当前最大的空闲分区容量时，即使主存中有空闲区域也不能够利用，从而降低了存储空间利用率。



四、页式存储管理

基本原理



四、页式存储管理

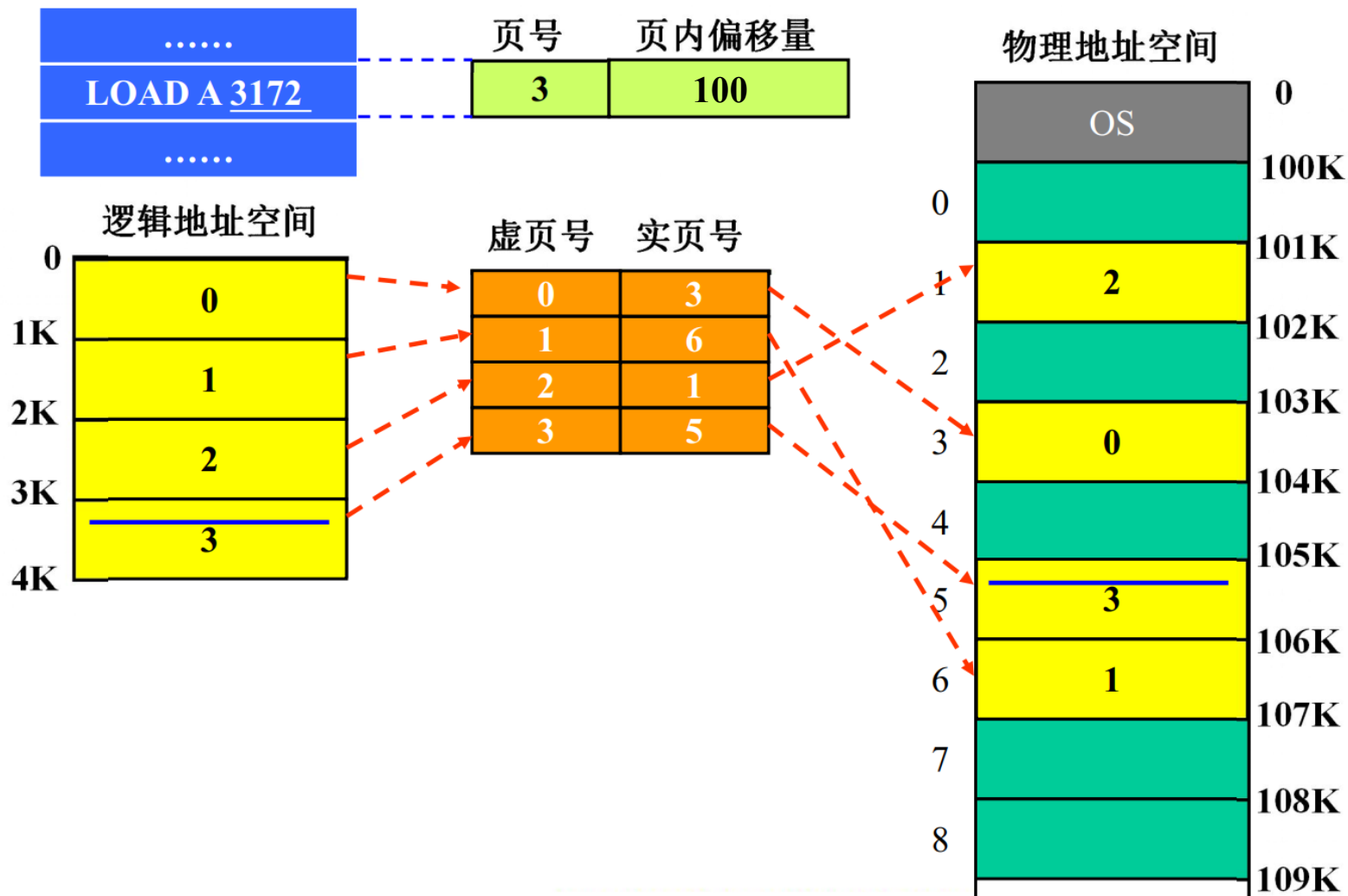
页面变换表(Page management table)

❖ 也叫页表：是一种特殊的数据结构。

用途：记录每一个作业的虚页号到物理内存中实页号之间的映射关系。

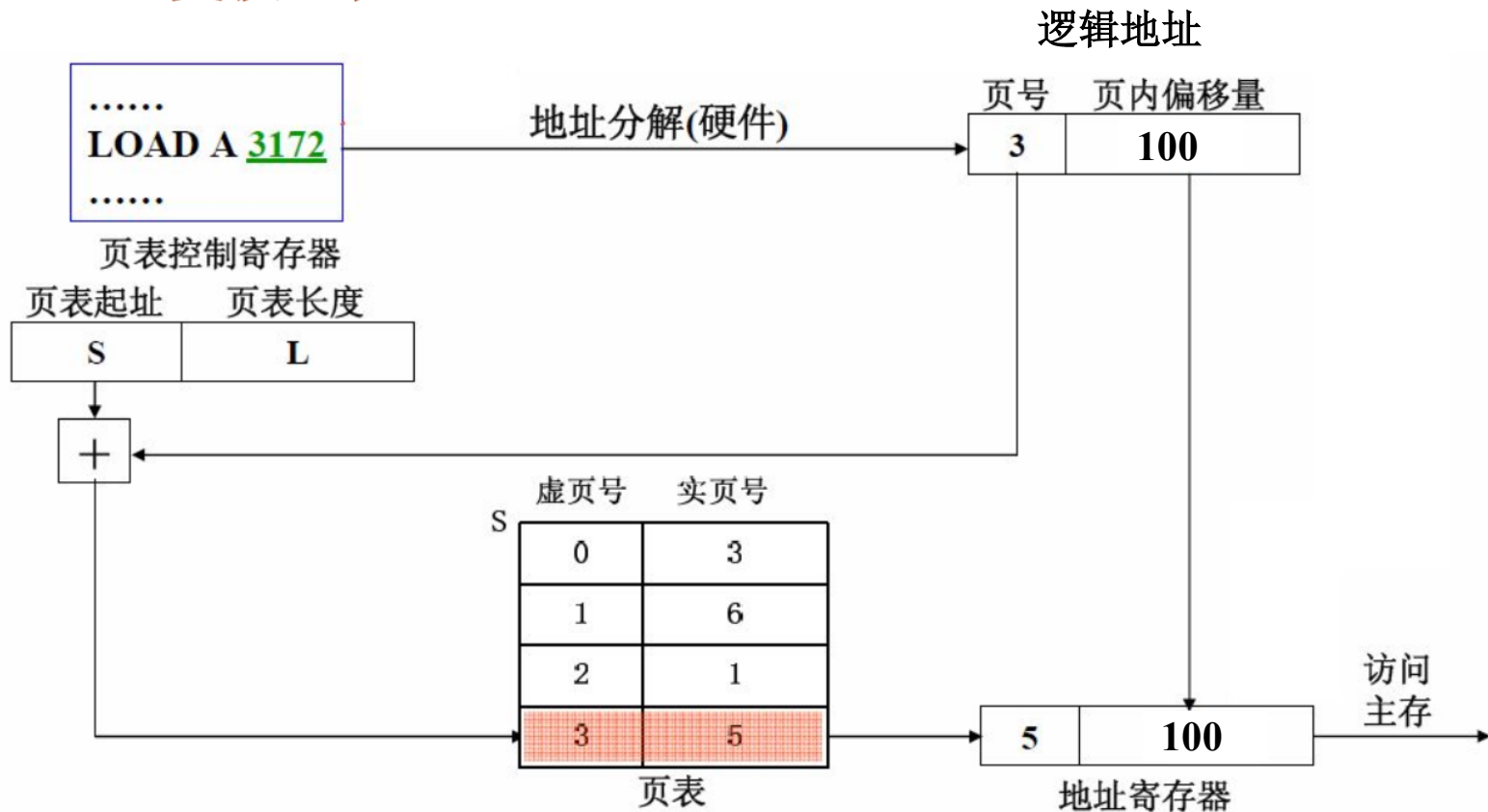
	虚页号 / 实页号	
Job1	0	3
	1	6
	2	1
	3	5
Job2	...	...
	...	...

四、页式存储管理



四、页式存储管理

地址变换过程



Q:为了取出一个数据，系统需要访问内存几次？

扩展思考：页式管理的数据结构和硬件支持

四、页式存储管理

优点

- ❖ 程序不必连续存放
- ❖ 没有外碎片

主要缺点

- ❖ 程序要一次全部装入内存
- ❖ 各种数据结构(页表)要占用一定的内存空间，而且系统要花费一定的时间来建立和管理这些表格
- ❖ 依然存在内碎片

五、段式存储管理

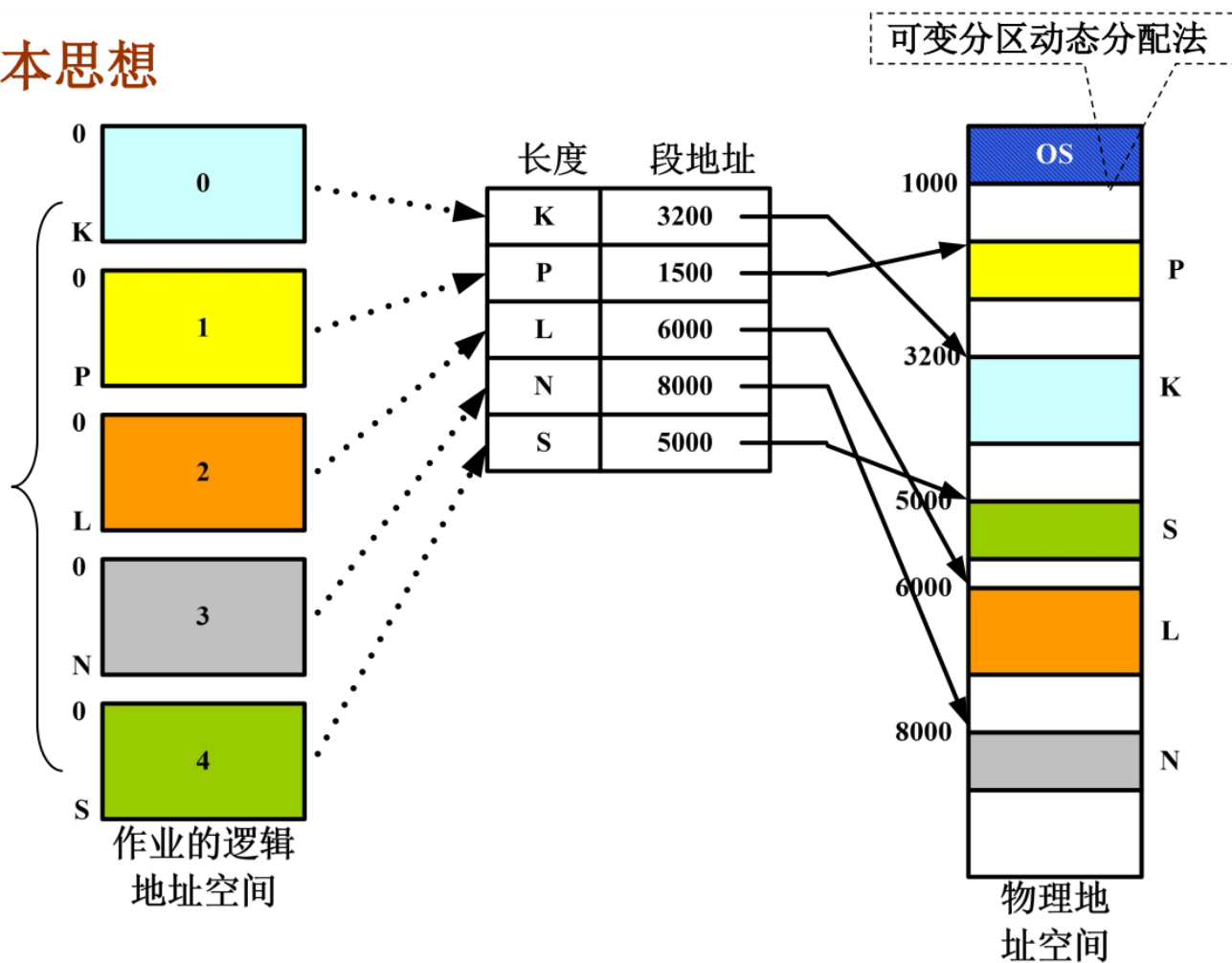
引入分段存储管理方式：

- ❖ 信息共享
- ❖ 信息保护
- ❖ 动态链接
- ❖ 动态增长

五、段式存储管理

基本思想

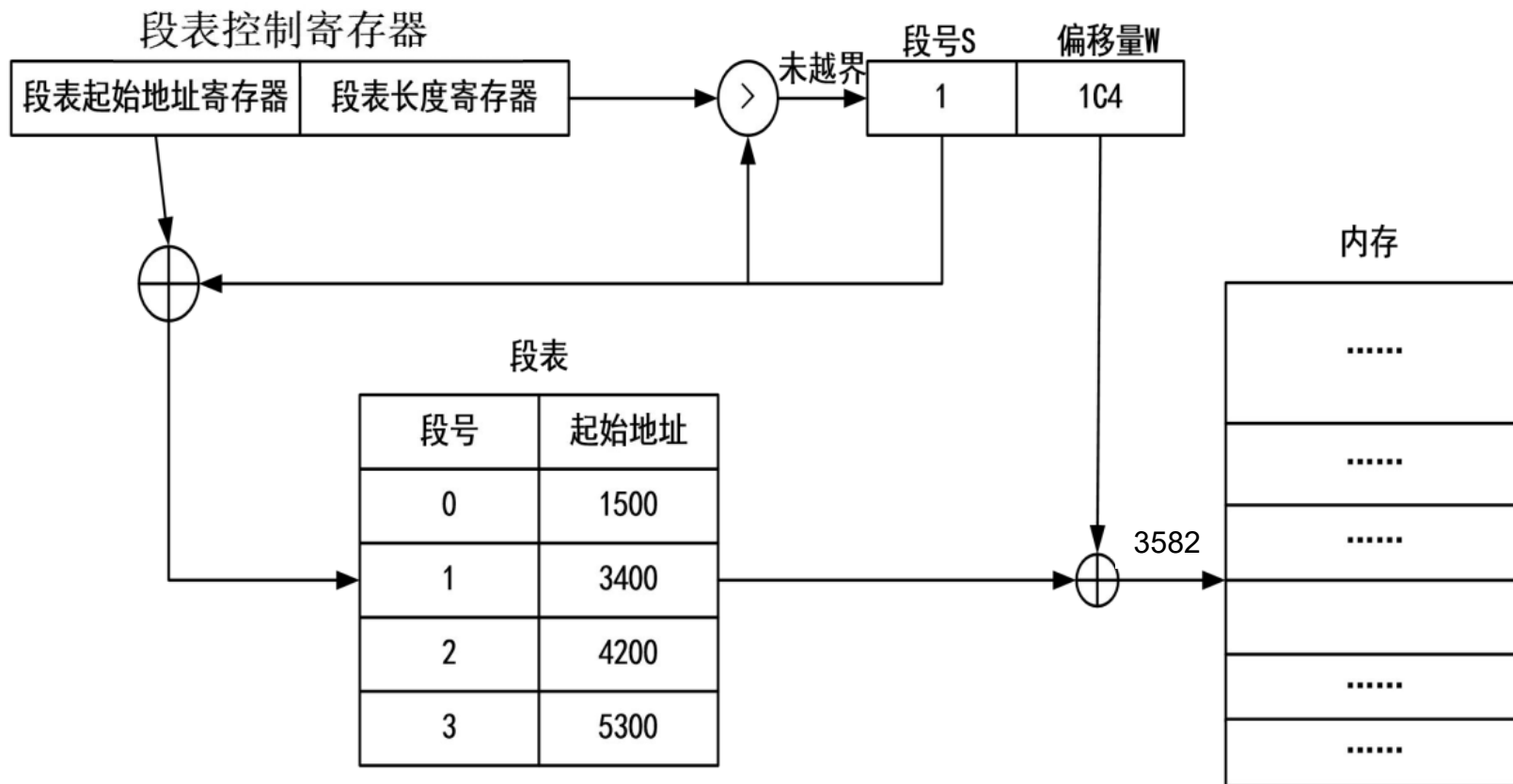
按照逻辑单位进行划分，每段拥有独立的逻辑空间。



五、段式存储管理



地址变换过程



五、段式存储管理



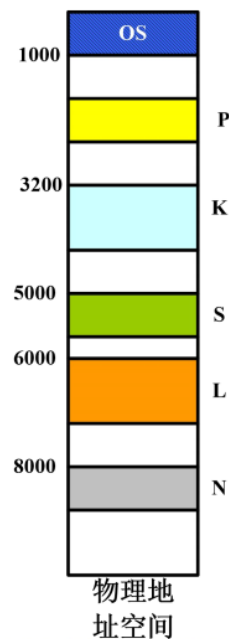
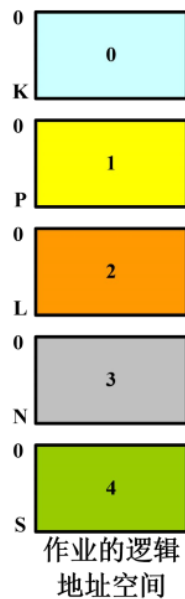
优点

- ❖ 程序不必连续存放
- ❖ 没有内碎片
- ❖ 便于实现共享，即允许若干个进程共享一个或者多个段



缺点

- ❖ 作业要一次全部装入内存
- ❖ 存在外碎片



五、段式存储管理

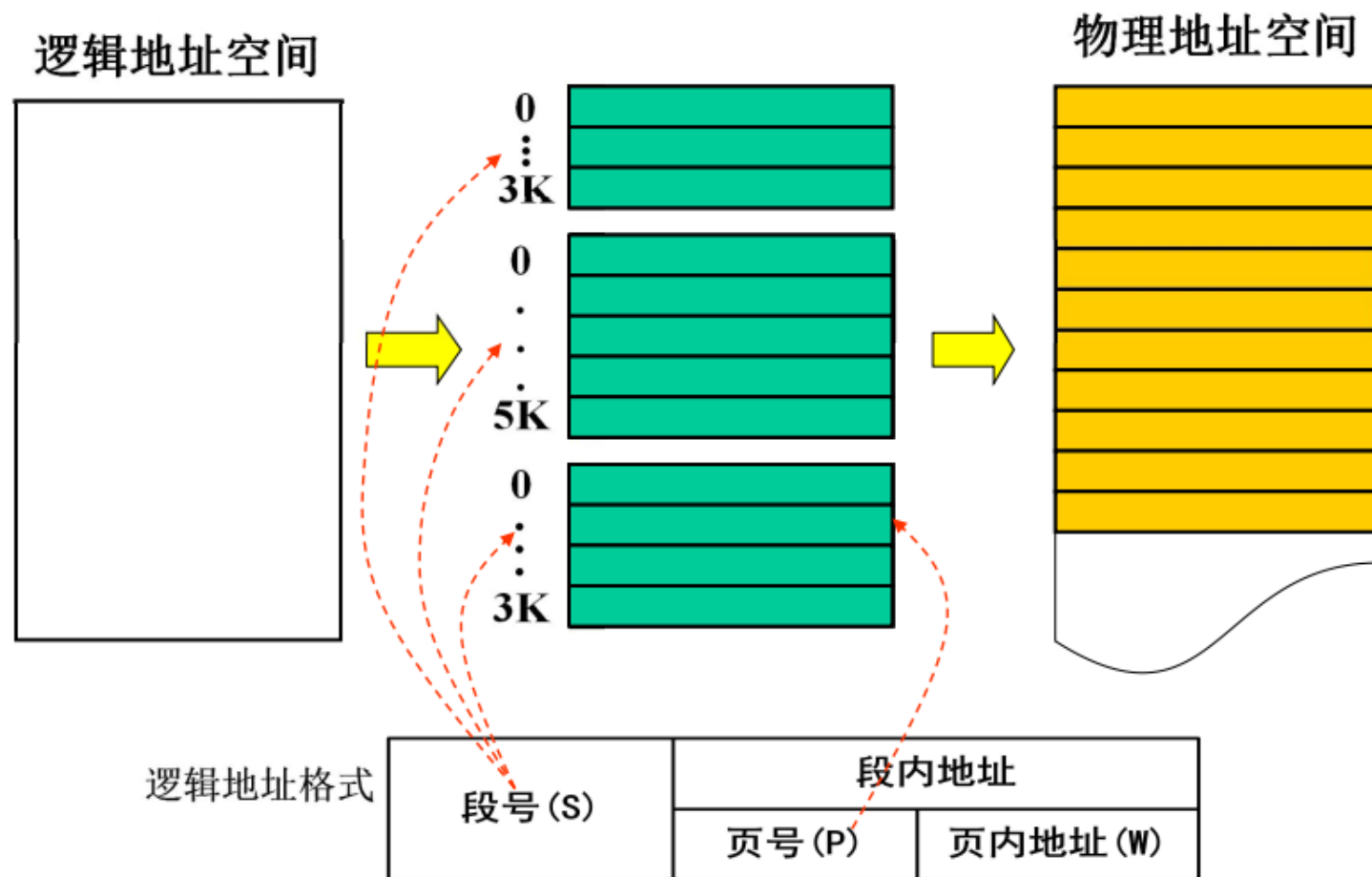
📌 分页式管理和分段式管理的比较

对比维度	页式存储管理	段式存储管理
设计目的	提高 内存利用率	满足 用户逻辑需求
划分依据	系统管理 需要	用户应用 需要
页/段大小	页大小 固定 （与程序逻辑无关）	段大小 不固定 （由模块逻辑决定，可长可短）
逻辑地址	单一 一维 线性地址（从 0 开始 连续编址 ，无逻辑分段）	多维 分段 独立编址 （每个段内地址从 0 开始，段间需“段名 + 段内地址”标识）
页表 / 段表	页面数量 多 →页表 长 ，查找耗时	段数量 少 →段表 短 ，查找速度快
碎片类型	内碎片 （页大小固定，页内剩余空间无法利用）	外碎片 （段大小不固定，内存分配后剩余分散空闲区，类似可变分区外碎片）
内存共享	不支持 （页面无逻辑关联）	支持 （段是逻辑模块，可直接共享完整功能段）
存储扩充	不支持 （页大小固定）	不支持 （作业需一次性装入所有段）
地址标识方式	逻辑地址 = 页号 + 页内偏移 （系统维度标识）	逻辑地址 = 段名（段号） + 段内偏移 （用户逻辑维度标识）
内存分配特点	以页为单位离散分配 ，无外碎片（但有内碎片），无需紧凑操作	以段为单位离散分配 ，产生外碎片（需紧凑操作解决，但紧凑耗时）

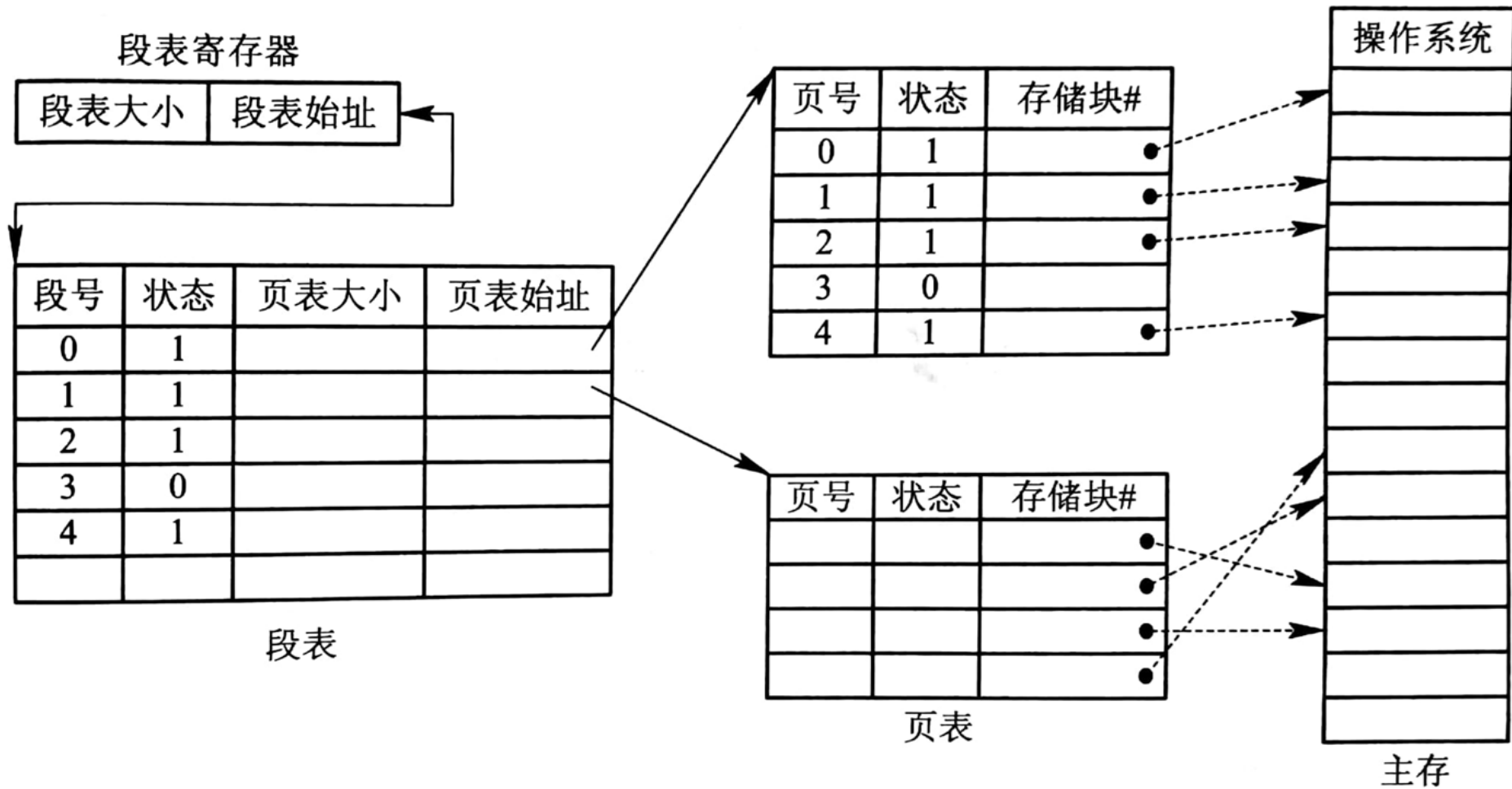
六、段页式存储管理



思想：



六、段页式存储管理

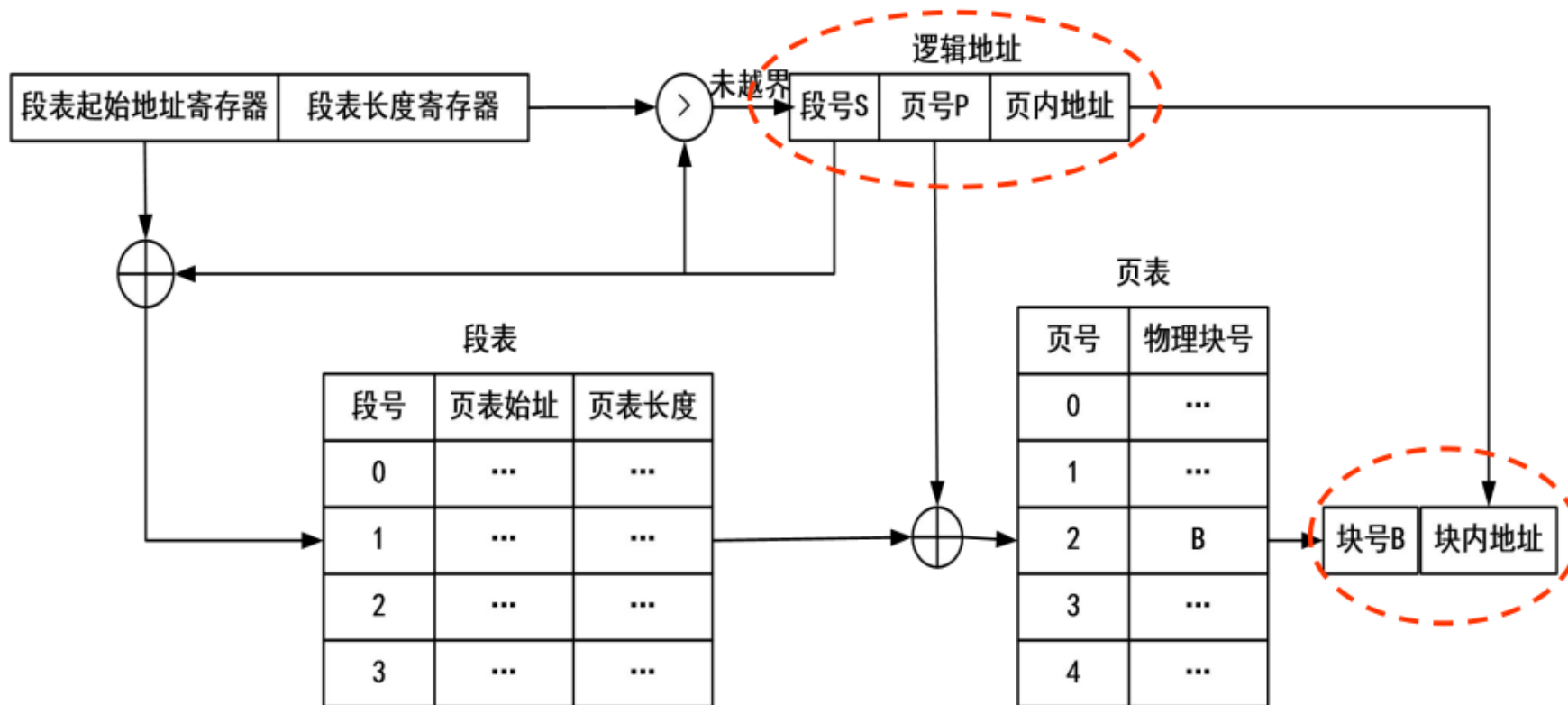


六、段页式存储管理



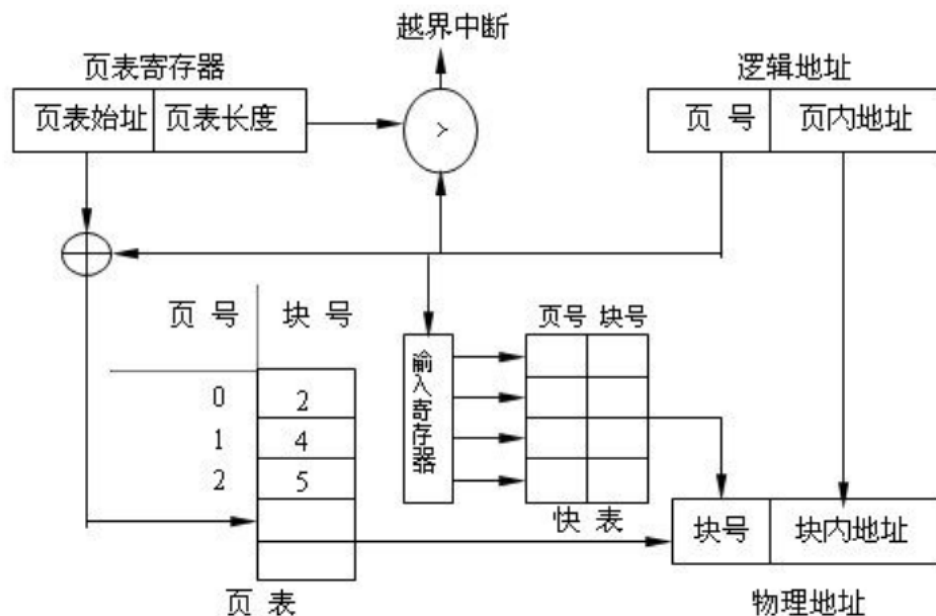
地址变换过程

Q: 为了获得一条指令或者数据, 需要访问内存几次?



* 快表

实际系统中会用 **TLB** (**快表** Translation Lookaside Buffer, 即转换旁路缓冲器) 优化, TLB是CPU内部的**高速缓存**, 存储最近用过的“段号+页号→物理块号”映射



- **页表**: 存完整的“页号→块号”映射 (内存中)
- **快表** (TLB): 存近期常用的“页号→块号”映射 (CPU高速缓存)
- **页表寄存器**: 存页表的起始地址和长度 (用于越界检查)

地址转换流程

- (1) **取逻辑地址**: 拆分出**页号**和**页内地址**
- (2) **查快表**: 用页号查快表→**若命中**, 直接取块号, 跳过页表访问
- (3) **查页表** (**快表未命中**时): 用页表寄存器的起始地址+页号, **访问**内存中的**页表**→取块号
- (4) **更新快表**: 若快表未命中, 将新的“页号→块号”映射存入快表 (**替换旧数据**)
- (5) **越界检查**: 页号与页表长度比较→超范围则触发“**越界中断**”
- (6) **合成物理地址**: 块号+块内地址→**访问物理内存**